

Using a Sandboxed Ticketing Bot to Teach Digital Trust and IT Audit*

Lorraine Lee^b, Hagit Levy-Shalev^a, and Hunter Ng^a

^a*Baruch College, City University of New York, United States*

^b*University of North Carolina Wilmington*

May 10, 2026

Abstract

The verification of Information Technology (IT) controls is a core responsibility of IT auditors. This case places students in the role of IT auditors assigned to assess the effectiveness of controls over a sandboxed online ticketing system. Students first participate in limited ticket drops, inspect the website's Document Object Model (DOM), and use AI-assisted coding to understand how automated purchasing can occur. Students then evaluate whether the system's controls over purchase limits, transaction authorization, backend logging, exception reporting, and monitoring are effective. The case introduces students to technical skills such as browser inspection, basic automation logic, and AI-assisted coding while maintaining an internal-control focus. Students document their work through a DOM discovery worksheet, ticket-drop observations, an internal-control review matrix, and a short reflection. Digital trust, automated transactions, IT controls, and AI-assisted technical fluency are all topics relevant to Accounting Information Systems (AIS), IT Audit, and Audit courses. As such, instructors can use this case at the undergraduate or graduate level in an Accounting Information Systems, IT Audit, or Audit class.

Keywords: IT General Controls, Digital Trust, Ticketing Bots, Internal Controls, AI-Assisted Coding, Accounting Information Systems

I. THE CASE

SeatGate

SeatGate

SeatGate is a sandboxed online ticketing system used by a university events office to distribute tickets for campus concerts, comedy shows, student-organization events, and selected city events. The system allows students to view upcoming events, select a ticket quantity, and submit a purchase request through a web browser. The events office uses SeatGate to manage limited ticket drops for high-demand events and to track completed purchases, remaining inventory, user activity, and exceptions.

SeatGate's management has engaged Rogers & Rogers LLP to perform an IT audit of the ticketing system. You are a junior IT auditor on the engagement team. Your senior manager, RR Rogers, has asked you to evaluate whether SeatGate's controls over online ticket purchases are designed and operating effectively. The engagement focuses on the system's application controls, access controls, logging controls, and monitoring procedures during limited ticket releases.

Management recently launched a limited online ticket drop for several high-demand events. Shortly after the release, students complained that tickets sold out almost immediately. Some users reported that they could not complete a purchase manually before the tickets disappeared. Others claimed that the website appeared to accept purchases faster than an ordinary user could reasonably click through the process. Management suspects that automated scripts may have been used to buy tickets faster than ordinary users.

SeatGate was designed to be a controlled transaction system. Each ticket purchase should be initiated by a valid user, checked against event-level purchase limits, processed against available inventory, recorded in the transaction log, and reflected in management reports. However, management is uncertain whether these controls operated as intended during the ticket drop. In particular, management is concerned about whether the website relied too heavily on front-end restrictions that could be inspected, bypassed, or manipulated by users with basic coding knowledge.

Rogers & Rogers LLP will use the COSO Internal Control–Integrated Framework and COBIT as reference points for the audit. Under COSO, the audit team will consider whether SeatGate's control environment, risk assessment, control activities, information and communication, and monitoring activities support reliable transaction processing. Under COBIT, the team will consider whether the system's IT processes support governance objectives such as access restriction, processing integrity, logging, exception monitoring, and management review. These frameworks provide a basis for evaluating whether SeatGate's policies are supported by actual system behavior and sufficient audit evidence.

The events office has the following stated policies:

1. Each user may purchase no more than one ticket per event.
2. Only valid users should be able to complete purchases.
3. Ticket inventory should update immediately after each sale.

4. Automated or suspicious purchasing behavior should be logged, and an exception report should be generated.
5. Every ticket purchased should have a timestamp in the admin view.

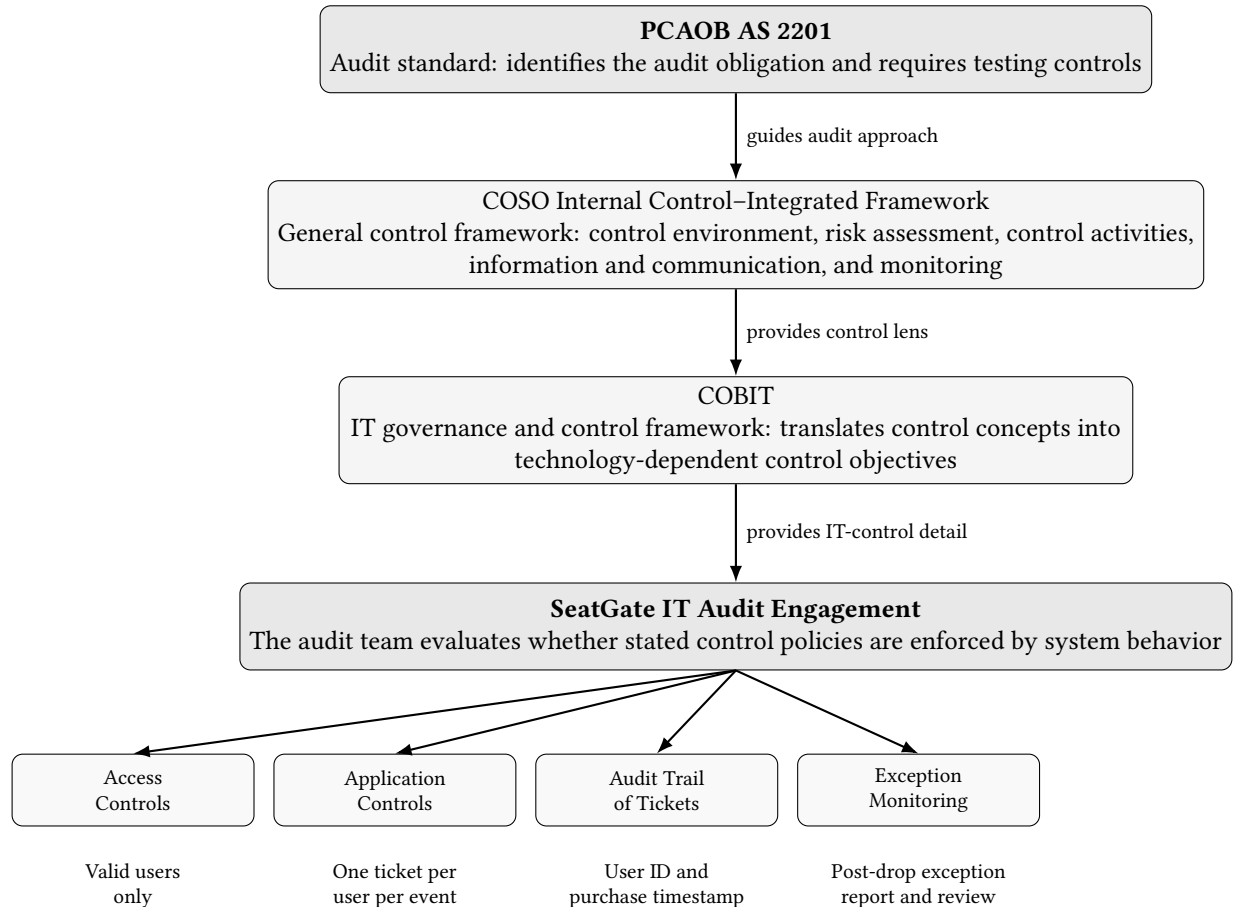


Figure 1: Relationship among PCAOB AS 2201, COSO, COBIT, and the SeatGate IT audit engagement

Figure 1 summarizes the relationship among the audit standard, the internal-control framework, the IT-control framework, and the SeatGate engagement. PCAOB AS 2201 motivates the audit task. COSO provides the broad internal-control lens. COBIT provides the IT-specific control lens used to evaluate access controls, application controls, logging, exception reporting, and monitoring.

Management has asked your audit team to determine whether the ticketing system’s controls are operating effectively. As part of your review, you will examine the website’s visible structure, observe how users interact with the purchase process, compare front-end behavior with backend records, and evaluate whether the system provides sufficient evidence for management to detect and respond to suspicious purchasing activity. You will then identify control weaknesses, assess their implications, and recommend improvements consistent with COSO and COBIT control principles.

Figure 2 summarizes the flow of your audit work. You begin as a junior IT auditor evaluating SeatGate’s ticketing system. The case is framed by AS 2201, with COSO providing the broad internal-control lens and

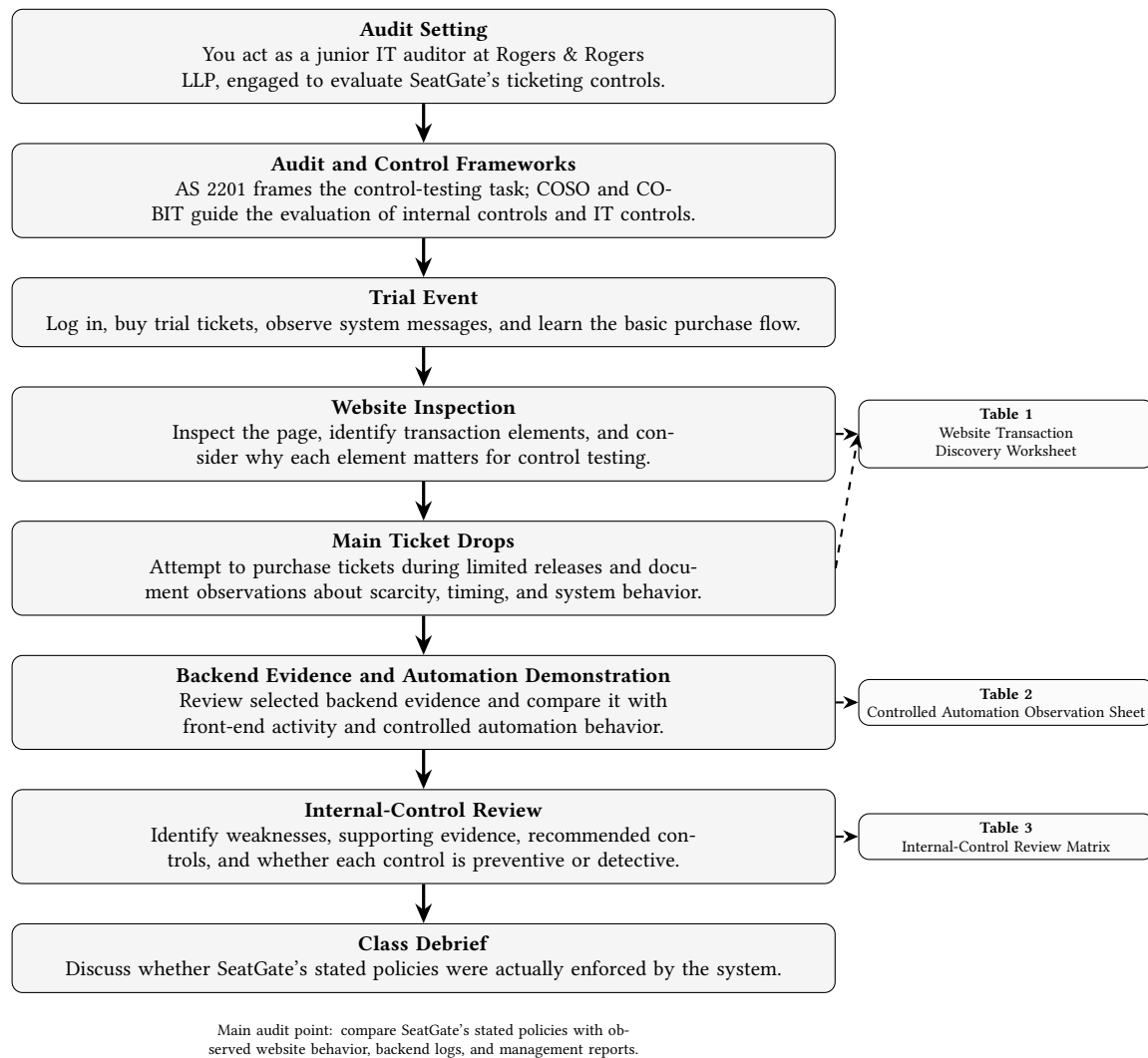


Figure 2: Flow of the SeatGate IT audit case

COBIT providing the IT-control lens. You then move from a trial event to website inspection, limited ticket drops, backend evidence, controlled automation observation, internal-control review, and class debrief.

Requirements

Before the main ticket drops begin, you will complete a trial event. The trial event includes three ticket waves and is designed to help you become familiar with the SeatGate purchase process. The trial event is not graded unless otherwise stated by your instructor. Log in using the username and password provided by your instructor. Practice viewing the event, selecting a ticket tier, submitting a purchase request, and observing the confirmation or error message.

During the trial event and the main ticket drops, inspect the SeatGate website and identify the parts of the webpage that support the purchase process. Document what you observed, explain what each page element does, and consider why each element matters for internal control. After the trial event, you will participate in three main ticket-drop events. Each main event includes three ticket waves. You are considered to have successfully purchased a ticket for an event if you purchase any one of the three ticket types for that event. Your objective is to observe the transaction process, attempt to purchase tickets within the stated rules, and document evidence relevant to the control review.

If your instructor permits AI-assisted coding, any code should be used only within the SeatGate sandbox and only for activities authorized by your instructor. Do not apply automation techniques to real ticketing websites or external systems. The purpose of the exercise is not merely to obtain tickets. Use the ticket drops to observe the transaction process, identify possible control deficiencies, and complete Table 1, the Website Transaction Discovery Worksheet. In particular, consider whether the system enforces purchase limits, validates users, logs suspicious behavior, updates inventory correctly, and provides management with timestamps and backend evidence for each ticket purchase.

Complete Table 1 during or immediately after the trial and actual events.

Table 1: Website Transaction Discovery Worksheet

How many tickets did you buy? Which event(s) did you buy tickets for?				
Page Area or Function	What You Observed on the Page	What the Page Element Does	Possible Internal-Control Issue	Audit Relevance
Home page		Introduces the ticketing website and directs users to available events or login functions	Users may misunderstand where the transaction begins or what actions are available	Helps auditors understand the entry point of the transaction process
Login page		Allows users to enter their email and password to access the system	Weak login controls may allow unauthorized users or bots to access accounts	Relevant to access control and user authentication
User account page		Displays available balance and tickets already purchased	Balance or purchase history may not match backend records	Relevant to access control, authorization, and completeness of transaction records

After completing the table, write a short process narrative:

A user begins by _____. The system then _____. A ticket purchase

is completed when _____.

After the trial event and the three main ticket-drop events, you will observe a controlled automation demonstration within the SeatGate sandbox. Compare the automated process with your own manual purchasing experience. You will also review selected backend evidence that management received after the ticket drops. This evidence may include transaction logs, failed purchase attempts, exception reports, user activity records, management reports, or other records relevant to the control review.

The automation demonstration shows how automated interaction can occur when a website exposes predictable page elements or does not adequately restrict rapid transaction attempts. The purpose of this demonstration is not to teach ticket scalping. The purpose is to show how automated behavior can expose weak digital controls and why IT auditors must understand both front-end activity and backend evidence.

Complete Table 2.

Table 2: Controlled Automation Observation Sheet

Question	Response
What did the script attempt to do?	
Did the system block the automated behavior?	
If you used AI to penetrate-test the controls, what did AI generate and what did you still need to understand yourself?	

Using your observations from the ticket drop, website inspection, automation demonstration, and backend evidence, identify control weaknesses and recommend improvements. This requirement should be completed in groups unless otherwise instructed.

Complete Table 3.

Table 3: Group Work: Internal-Control Review

Observed Weakness	Recommended Control	Control Type

Be prepared to discuss one observed weakness, the evidence supporting that weakness, and your recommended control. In the class debrief, connect your observations to the audit engagement. Consider whether a transaction can be recorded by the system but still fail to satisfy the underlying control objective. You should also consider whether SeatGate's controls enforced purchase limits, validated users, updated inventory correctly, logged suspicious behavior, and produced exception reports that management could review.

As part of the class discussion, address the following questions:

1. What was the most important internal-control weakness revealed by the ticket-drop exercise?

2. What evidence would an IT auditor need to determine whether management's stated policies were actually enforced?

Motivation for the Case

The Sarbanes-Oxley Act of 2002 (SOX) requires management to maintain and assess internal controls over financial reporting. Under PCAOB AS 2201, auditors evaluate these controls using a top-down approach that begins with financial statement risks and then focuses on the controls most relevant to material misstatement. Information Technology (IT) general controls are central to this process because they support the reliability of application controls, transaction processing, and system-generated data.

IT auditors examine whether information systems are designed, secured, operated, and monitored in a way that supports reliable data and transaction processing (Green, Box, and Okas, 2026). IT controls are therefore a basic component of IT audit, including controls over user access, system changes, application processing, system-generated reports, and audit trails (Lee and Sawyer, 2019). While these topics are important, contemporary IT auditors must also understand automated transactions, platform governance, data analytics, cybersecurity controls, artificial intelligence, and the risks created by rapidly changing digital systems (Dzuranin and Mălăescu, 2016; Li and Goel, 2025).

Technical fluency is therefore becoming part of audit competence. Coding is increasingly relevant to accountants because it allows professionals to automate repetitive tasks, inspect data and systems directly, and understand the logic embedded in digital business processes (ACCA, 2021). Recent professional commentary similarly notes that accountants are already using AI for automation, data analysis, technical research, and document preparation (Kenney, 2025). These developments suggest that auditors need a certain level of coding and AI literacy to evaluate system behavior, validate automated outputs, and communicate effectively with technical specialists.

A challenge for AIS instructors is to teach these technical skills without turning the course into a computer science course. The instructional setting must be simple enough for students to understand quickly, but rich enough to show how system design affects internal controls. We use online ticketing as this setting. Students understand the business process, the scarcity of tickets creates an intuitive incentive for automation, and the transaction flow allows instructors to connect technical behavior to audit concepts. The *Wall Street Journal* has reported on ticket scalping, resale practices, and ticketing fees, reflecting a broader phenomenon that students are likely to recognize (Steele, 2025; Fanelli and Michaels, 2026). Using a real-life example gives the case verisimilitude and engages students, while allowing instructors to keep the implementation focused on internal controls, digital transaction processes, and basic coding.

We refer to a prominent incident that happened in September 2025, when the Federal Trade Commission, joined by seven states, sued Live Nation and Ticketmaster for allegedly engaging in illegal ticket resale practices and deceiving artists and consumers about prices and ticket limits. The FTC alleged that Ticketmaster publicly claimed to protect fans from brokers while benefiting from brokers who used thousands of accounts and proxy IP addresses to bypass stated ticket limits, acquire large volumes of tickets, and resell them on Ticketmaster's own resale platform at substantial markups (Federal Trade Commission, 2025).

These allegations provide a useful setting for teaching accounting information systems because they show that ticketing failures are not only consumer-protection or cybersecurity problems. They are also internal-control problems. A ticketing platform may record a sale correctly, but the transaction may still violate purchase limits, platform rules, artist restrictions, or anti-bot requirements. In that case, the data may be mechanically complete but economically misleading.

From an IT audit perspective, the allegations raise direct control questions. Were purchase limits actually enforced? Did the company monitor high-volume brokers, repeated purchases, proxy activity, or unusual account behavior? Were exceptions reviewed and remediated? Did resale revenues depend on activity that violated the company's stated rules? These questions connect bot activity to core audit concepts, including authorization, transaction validity, monitoring, access control, and management override.

Hence, we present a case which uses a sandboxed classroom ticketing website to teach IT audit and internal-control concepts through a controlled ticket-drop exercise. Students first participate in three limited ticket releases as ordinary users. They then inspect the website's HTML structure and Document Object Model (DOM), use AI-assisted "vibe coding" to understand automated interaction, and test whether the platform's controls prevent, detect, or correct bot-like purchasing behavior. The purpose is not to teach ticket scalping, but to show how digital transactions are initiated, authorized, processed, logged, monitored, and reported. After the exercise, the instructor reveals backend logs and the automation script, allowing students to compare front-end behavior with system-recorded activity. Students then identify control weaknesses, evaluate audit evidence, and recommend preventive and detective controls.

The case uses gamification (Blix, Edmonds, Edmonds, and Sorensen, 2026) and a real-world ticket-scalping context (Gantman, 2024) to connect coding, transaction processing, backend logging, and internal-control reasoning. By requiring students to inspect the DOM, observe automated transactions, analyze exception reports, and prepare audit documentation, the case responds to calls for AIS education to integrate technical fluency, AI-related skills, and audit judgment (AICPA, 2025; Dzuranin and Mălăescu, 2016; Li and Goel, 2025; ACCA, 2021; Kenney, 2025).

Past Literature on IT Controls

Following the structure of Lee and Sawyer (2019), we review recent instructional cases, in the last five years, related to internal controls and IT controls from three leading accounting education journals: *Issues in Accounting Education*, *Journal of Accounting Education*, and *AIS Educator Journal*. Table 4 summarizes the cases most closely related to this study.

Table 4: Recent instructional cases on internal controls and IT controls (2021–2026)

Year	Citation	Full title	Primary topics & software	Main task related to IT control
2021	Lawson and Street (2021)	Detecting dirty data using SQL: Rigorous house insurance case	Data quality, claims analytics, preventative IS controls; software: SQL	Query claims data, identify integrity exceptions, and recommend preventative information-system controls.
2022	Boss et al. (2022)	Accountants, Cybersecurity Isn't Just for "Techies": Incorporating Cybersecurity into the Accounting Curriculum	Cybersecurity, disclosure, audit/tax risks, short cases; artefacts: case materials and 10-K-style disclosures	Analyse six cyber scenarios and evaluate risk, control, reporting, and assurance implications.
2022	Brown and Lightle (2022)	Only Reliable Parts and Supplies, Inc.: Assessing and documenting the design of internal controls	Sales-process controls, flowcharting, audit documentation; software: flowcharting/word-processing tools	Flowchart the sales cycle, assess control design, and draft a memo on internal-control weaknesses.
2023	Parlier and Lee (2023)	Inventory analytics: A teaching case using excel and Alteryx	Audit analytics, inventory anomalies, exception analysis; software: Excel and Alteryx	Use analytics to investigate inventory anomalies and discuss implications for control monitoring and audit evidence.
2023	Hawk (2023)	Building Higher-Order Thinking Skills Through Assessment of User Account Access	IT general controls, user access management, audit analytics; software: Excel and CaseWare IDEA	Test user-access controls, analyse exceptions, and conclude on ITGC effectiveness.
2024	Boss et al. (2024)	Be an Expert: A Critical Thinking Approach to Responding to High-Profile Cybersecurity Breaches	Cyber breaches, internal audit, incident response; software: case materials	Evaluate breach causes, prevention, and organisational response from security, CIO, and internal-audit perspectives.
2024	Soonawalla and Wakefield (2024)	Stopping Hamburglars: Applying Effective Internal Control	COSO, fraud triangle, business-process controls; software: none stated	Diagnose control weaknesses and redesign a stronger internal-control system for a fast-food operation.
2024	Zeller et al. (2024)	Workbook design and controls: A framework	Spreadsheet controls, workbook design, reviewability; software: Excel	Redesign a workbook using control principles that improve reliability, transparency, and review efficiency.
2024	Mascha and Miller (2024)	Applying Internal Control Concepts Using Database Design: An Educational Case	Internal controls, transaction processing, database design; software: Microsoft Access	Build database tables, forms, queries, and reports that embed internal-control concepts in system design.
2024	Jiang et al. (2024)	The Sunday Standstill: An Accounting Information System Upgrade Role-Play	AIS implementation, change management, information security policy, internal controls; software: role-play case	Evaluate upgrade decisions and control consequences during an AIS change scenario.
2025	Bloch et al. (2025)	The Falling House of Wirecard	Fraud, governance, internal-control failure in digital payments; software: none stated	Analyse red flags, governance breakdowns, and internal-control weaknesses behind a digital-platform collapse.
2026	Janvrin et al. (2026)	Colonial Pipeline: Responding to a Ransomware Attack in the Energy Sector	Ransomware, cyber governance, contingency planning, resilience; software: none stated	Assess incident response, cyber-control weaknesses, and resilience planning after a major ransomware attack.

Note: Entries are limited to studies from *Issues in Accounting Education*, *Journal of Accounting Education*, and *AIS Educator Journal* (2021–2026) whose public publisher metadata put them as teaching cases, educational cases, case reports, role-plays, or paired teaching-note items.

Important Limitation

This exercise is restricted to the sandboxed classroom website. Students should not use automation techniques on real ticketing websites, commercial platforms, or systems that they do not own or have explicit written permission to test. Unauthorized automation, circumvention of access controls, credential misuse, scraping, or interference with commercial systems may violate website terms of service and applicable civil or criminal laws, including the Computer Fraud and Abuse Act and, in the ticketing context, the BOTS Act. The purpose of the exercise is to understand IT audit risk in a controlled environment, not to conduct unauthorized automation.

Learning Objectives

After completing the case, students should be able to:

1. Identify the main steps in a digital ticket-purchase transaction.
2. Explain how automated purchasing can create internal-control risks.
3. Distinguish between a transaction that is recorded and a transaction that is properly authorized.
4. Evaluate whether purchase-limit, authorization, logging, monitoring, and exception-reporting controls are operating effectively.
5. Recommend preventive and detective controls to reduce bot-like purchasing behavior.
6. Prepare audit documentation that connects observed system behavior to internal-control findings.

Instructor Implementation Guidance

Several materials accompany this case, including an instructional video, a PowerPoint deck for the student debrief, and student worksheets. These materials are available in the case GitHub repository at <https://github.com/hunternbh/acc3202-ticketing-bot>. The instructional video is also available at <https://youtu.be/gP3dJlWociM>.

SeatGate is designed as a sandboxed IT audit case in which students act as junior IT auditors evaluating whether the system's stated control policies are actually enforced by system behavior and supported by audit evidence. The case connects AS 2201, COSO, and COBIT to a live digital transaction process involving authentication, ticket inventory, purchase authorization, logging, exception monitoring, and backend evidence. The accompanying materials are provided to help instructors implement the exercise, guide student work, and debrief the case after the ticket-drop activity.

SeatGate is designed as a sandboxed IT audit case in which students act as junior IT auditors evaluating whether the system's stated control policies are actually enforced by system behavior and supported by audit evidence. The case connects AS 2201, COSO, and COBIT to a live digital transaction process involving authentication, ticket inventory, purchase authorization, logging, exception monitoring, and backend

evidence. A GitHub repository and instructional video are provided to instructors as part of the teaching materials.

Before the main ticket drops begin, the instructor should seed the database. Seeding refreshes the exercise data, resets previous purchases, recreates the user accounts, and creates the trial event. For the trial event, three ticket waves are created, and 999 tickets are available in each wave. The trial event is intended to be low-stakes. Students should use it to learn the transaction flow, inspect the website, and test any instructor-approved AI-assisted code. The trial event should not normally be graded, although instructors may use their discretion.

After the trial event, the instructor should conduct three main ticket-drop events. Each main event should include three ticket waves. A student is considered to have successfully purchased a ticket for an event if the student purchases any one of the three ticket types for that event. To generate student interest, instructors may decide whether ticket purchases count toward the course or case grade. We recommend allocating approximately 10 percent of the case grade to this component. Under this approach, each student begins with a \$5 budget, and the objective is to purchase one ticket from each of the three main events.

The instructor should release tickets gradually using the provided backend commands. Releasing tickets slowly allows students to observe scarcity, timing pressure, ordinary user behavior, and rapid purchasing behavior. The instructor may release fewer tickets than the number of students in the class to create a realistic limited-release environment. For example, if there are N students, each event may have approximately $N - 20\%$ tickets available.

Students may be allowed to use AI-assisted coding to understand how the website works, automate parts of the purchase process in the sandboxed environment, or test whether SeatGate’s controls prevent or detect rapid purchase attempts. Instructors should clearly state that this activity is limited to the SeatGate sandbox. The purpose of the exercise is not to teach ticket scalping or circumvention of real ticketing systems. The purpose is to show how automated behavior can expose weak digital controls and why IT auditors must understand both front-end behavior and backend evidence.

After the ticket drops, the instructor should reveal selected backend reports. These may include transaction logs, failed purchase attempts, exception reports, user activity records, ticket ownership records, wallet balances, or other evidence relevant to the control review. The instructor may also demonstrate a controlled automation script. The demonstration should help students compare manual purchasing behavior with automated purchasing behavior. Students should observe whether the system blocks rapid activity, logs suspicious behavior, enforces purchase limits, updates inventory correctly, and produces evidence that management could review.

Table 6 provides one possible implementation structure.

Student performance can be assessed using both technical and audit-based deliverables. The ticket-drop result provides evidence that students engaged with the website and understood how digital transactions can be affected by timing, automation, and control design. However, the main grading emphasis should be placed on the quality of students’ audit reasoning. Instructors may choose to make the ticket-purchase outcome a small portion of the grade or treat it as participation only. This reduces the risk that students focus on “winning” the ticket drop rather than understanding the control implications.

Table 6: Implementation Guidance

Phase	Description	Who Performs Phase	Estimated Time
Phase 1	Introduce the SeatGate audit setting, sandbox limitation, stated control policies, and the AS 2201, COSO, and COBIT framing.	Instructor	15–20 minutes
Phase 2	Students complete the trial event, inspect the website transaction process, and begin the Website Transaction Discovery Worksheet.	Students	25–35 minutes
Phase 3	Students participate in the main ticket drops and continue documenting observations about purchase behavior, inventory, limits, and system messages.	Students	30–45 minutes
Phase 4	Instructor demonstrates controlled automation and back-end evidence, including logs, failed attempts, or reports relevant to control testing.	Instructor/class	25–35 minutes
Phase 5	Students complete the internal-control review in groups and recommend preventive or detective controls.	Students/groups	30–45 minutes
Phase 6	Instructor leads the class debrief and connects observations to audit evidence, authorization, transaction validity, logging, and monitoring.	Instructor/class	20–30 minutes

The case may be assessed using two sources of evidence: a required case worksheet and a post-case perception survey. The worksheet asks students to document their observations from the ticketing website, the ticket-purchase process, the automation demonstration, and the backend evidence provided by the instructor. These responses allow the instructor to assess whether students can connect technical observations, such as rapid purchasing, exposed page elements, purchase-limit behavior, and backend logs, to audit concepts such as authorization, transaction validity, monitoring, exception reporting, and server-side validation. The post-case perception survey asks whether the exercise improved students' understanding of IT audit, internal controls, automated purchasing risks, AI-assisted coding, and the importance of technical fluency for auditors.

Table 7: Suggested Grading Allocation

Component	Suggested Weight
Participation in ticket drops and sandbox activity	10%
Website Transaction Discovery Worksheet	30%
Controlled Automation Observation Sheet	20%
Internal-Control Review Matrix	30%
Post-case reflection or survey completion	10%

The instructor debrief should emphasize that a transaction can be recorded by the system but still fail to satisfy the underlying control objective. Front-end restrictions are not sufficient if important controls are not enforced on the server side. A valid audit trail should include enough information to evaluate user

identity, timing, authorization, inventory effects, and exception behavior. Automated purchasing risk is not only a technical issue; it is also an internal-control, monitoring, and governance issue. COSO provides the broad internal-control lens, while COBIT helps translate that lens into IT-control objectives such as access restriction, processing integrity, logging, exception monitoring, and management review.

The SeatGate environment is a sandboxed classroom system. Instructors should remind students that automation should not be used on real ticketing websites or external systems. The case should be framed as an IT audit and internal-control exercise, not as an exercise in ticket scalping. The appropriate learning objective is to understand how digital transactions are initiated, authorized, processed, logged, monitored, and reported.

Case Effectiveness

We assess case effectiveness using two sources of evidence: a required case worksheet and a post-case perception survey. This approach follows prior AIS case research that evaluates student learning and student perceptions of case usefulness ([Lee and Sawyer, 2019](#)).

First, students complete a case worksheet during and after the exercise. The worksheet asks students to document their observations from the ticketing website, the ticket-purchase process, the automation demonstration, and the backend evidence provided by the instructor. These responses allow the instructor to assess whether students can connect technical observations, such as rapid purchasing, exposed page elements, purchase-limit behavior, and backend logs, to audit concepts such as authorization, transaction validity, monitoring, exception reporting, and server-side validation.

Second, students complete a short post-case perception survey. The survey asks whether the exercise improved their understanding of IT audit, internal controls, automated purchasing risks, AI-assisted coding, and the importance of technical fluency for auditors. The survey provides evidence on students' perceived learning and the usefulness of the case as an instructional activity.

Student Feedback

After completing the case, students complete a short perception survey using a seven-point Likert scale, where 1 = strongly disagree and 7 = strongly agree. Table 8 reports the survey items.

The survey also includes two open-ended questions: (1) What was the most useful part of the case? and (2) What change would improve the case for future students?

Teaching Notes

Teaching notes are included in the Appendix. The teaching notes provide instructor setup instructions, suggested classroom commands, guidance for releasing tickets, sample internal-control weaknesses, and suggested debriefing points. Instructors should not distribute the teaching notes to students before the exercise.

Table 8: Post-Case Student Perception Survey

Item	Statement
Q1	As a result of the case, I have a better understanding of how to test IT controls in a digital transaction system.
Q2	As a result of the case, I better understand how automated purchasing can create internal-control risks.
Q3	As a result of the case, I better understand the difference between a recorded transaction and an authorized transaction.
Q4	As a result of the case, I improved my ability to inspect basic HTML or webpage elements.
Q5	As a result of the case, I better understand how AI-assisted coding can help auditors examine digital systems.
Q6	As a result of the case, I better understand the risks of relying on AI-generated code without validation.
Q7	The internal-control review helped me connect technical observations to audit findings.
Q8	The case was a positive learning experience.
Q9	The case was an interesting assignment.
Q10	I recommend continuing to use this case with accounting students.

Suggested Answer Key

References

- ACCA (2021). Coding: as a professional accountant, why you should be interested. Technical report.
- AICPA (2025). AICPA and CIMA Launch AI Accelerator Skills Program To Prepare Finance Leaders for an AI Enabled Future.
- Blix, L., C. Edmonds, J. Edmonds, and K. Sorensen (2026). Social Gamification in Online Accounting Courses and the Impact on Course Engagement. *Issues in Accounting Education* 41(2), 1–27.
- Bloch, R. I., K. E. Hunter, and G. Kleinman (2025). The falling house of wirecard. *Issues in Accounting Education* 40(2), 121–132.
- Boss, S. R., J. Gray, and D. J. Janvrin (2022). Accountants, cybersecurity isn’t just for “techies”: Incorporating cybersecurity into the accounting curriculum. *Issues in Accounting Education* 37(3), 73–89.
- Boss, S. R., J. Gray, and D. J. Janvrin (2024). Be an expert: A critical thinking approach to responding to high-profile cybersecurity breaches. *Issues in Accounting Education* 39(1), 93–121.
- Brown, K. F. and S. Lightle (2022). Only reliable parts and supplies, inc.: Assessing and documenting the design of internal controls. *Journal of Accounting Education* 60, 100783.
- Dzurandin, A. C. and I. Mălăescu (2016). The current state and future direction of IT audit: Challenges and opportunities. *Journal of Information Systems* 30(1), 7–20.
- Fanelli, J. and D. Michaels (2026, March). Live Nation CEO Grilled Over Ticket Fees at Antitrust Trial. *Wall*

Street Journal.

- Federal Trade Commission (2025, September). FTC Sues Live Nation and Ticketmaster for Engaging in Illegal Ticket Resale Tactics and Deceiving Artists and Consumers about Price and Ticket Limits. Last Accessed May 4, 2026.
- Gantman, S. (2024). Does Using Real Organizations for Team Projects in an AIS Class Make a Difference? *AIS Educator Journal* 19(1), 17–34.
- Green, S., D. Box, and J. Okas (2026). IT Audit Services. https://www.ey.com/en_us/services/technology-risk/information-technology-audit-services.
- Hawk, H. (2023). Building higher-order thinking skills through assessment of user account access. *AIS Educator Journal* 18(1), 39–49.
- Janvrin, D. J., S. R. Boss, and J. Gray (2026). Colonial pipeline: Responding to a ransomware attack in the energy sector. *Issues in Accounting Education* 41(2), 131–145.
- Jiang, R., M. Luttenton-Knoll, P. Hillman, and D. A. Pellathy (2024). The sunday standstill: An accounting information system upgrade role-play. *AIS Educator Journal* 19(1), 50–69.
- Kenney, B. A. (2025, June). Real-life ways accountants are using AI.
- Lawson, J. G. and D. A. Street (2021). Detecting dirty data using SQL: Rigorous house insurance case. *Journal of Accounting Education* 55, 100714.
- Lee, L. and R. Sawyer (2019). IT general controls testing: Assessing the effectiveness of user access management. *AIS Educator Journal* 14(1), 15–34.
- Li, Y. and S. Goel (2025). Bridging IT auditors and AI auditing: Understanding pathways to effective IT audits of AI-driven processes. *Advances in Accounting* 69, 100842.
- Mascha, M. F. and C. L. Miller (2024). Applying internal control concepts using database design: An educational case. *AIS Educator Journal* 19(1), 1–16.
- Parlier, J. and L. Lee (2023). Inventory analytics: A teaching case using excel and alteryx. *Journal of Accounting Education* 63, 100848.
- Soonawalla, K. and J. Wakefield (2024). Stopping hamburglars: Applying effective internal control. *Issues in Accounting Education* 39(4), 165–181.
- Steele, A. (2025, March). Trump Signs Order Targeting Ticket Scalpers and Fees. *Wall Street Journal*.
- Zeller, T., E. Dingrando, and D. Booker (2024). Workbook design and controls: A framework. *Journal of Accounting Education* 69, 100933.

Appendix

A How to Set Up the Site

The instructional site is hosted in the GitHub repository:

<https://github.com/hunternbh/acc3202-ticketing-bot>

The public teaching page at <https://hunterng.com/teaching> provides the classroom commands for reseeding the database, releasing tickets, checking holdings, checking ticket availability, and running the bot program. An instructional video is also available there. This section provides setup instructions for instructors who are not familiar with GitHub, Render, or command-line tools.

A.1 Overview of the Classroom Setup

The exercise uses a sandboxed ticketing website. Before class begins, Event 1 has tickets priced at \$0. The remaining four events have tickets priced at \$1 each. The instructor should update the list of student users in the backend before deploying the site. The user list is stored in the backend seed file:

`server/seed.js`

The deployed backend runs on Render. Because the free Render server can become inactive when it is not used, the instructor can set up a scheduled refresh that visits the backend every five minutes during the class period. This keeps the classroom site responsive.

A.2 Downloading the Code from GitHub

There are two ways to download the code.

Option A: Download as a ZIP File

This is the easiest method for instructors who do not use Git.

1. Open <https://github.com/hunternbh/acc3202-ticketing-bot>.
2. Click the green **Code** button.
3. Click **Download ZIP**.
4. Save the ZIP file to the computer.
5. Right-click the ZIP file and choose **Extract All**.
6. Open the extracted folder.

Option B: Clone with Git

If Git is installed, open PowerShell or Terminal and run:

```
git clone https://github.com/hunternbh/acc3202-ticketing-bot.git
cd acc3202-ticketing-bot
```

A.3 Changing the Student User List

Before deploying the site, edit the student accounts in:

```
server/seed.js
```

The seed file contains a list of users similar to the following:

```
const users = [
  { email: 'student01@hunter-ticket.com', password: 'pass01', isAdmin: false },
  { email: 'student02@hunter-ticket.com', password: 'pass02', isAdmin: false },
  { email: 'admin@hunter-ticket.com', password: 'adminpass', isAdmin: true },
]
```

Replace or expand the student list so that each student has an assigned login. For example, for a class of 70 students, create 70 student accounts plus one admin account. Keep the admin account separate from student accounts.

A.4 Deploying the Backend on Render

The backend runs from the server folder. The instructor should deploy this folder as a Render Web Service.

1. Create or sign in to a Render account at <https://render.com>.
2. In the Render dashboard, click **New**.
3. Choose **Web Service**.
4. Connect the GitHub repository.
5. Select the repository `acc3202-ticketing-bot`.
6. Use the following settings:

Table 9: Render Backend Settings

Setting	Value
Service type	Web Service
Repository	hunternbh/acc3202-ticketing-bot
Branch	main
Root directory	server
Runtime	Node
Build command	npm install
Start command	npm start
Instance type	Free or paid, depending on use

A.5 Creating the Database

The backend uses PostgreSQL. In Render, create a PostgreSQL database and connect it to the backend.

1. In the Render dashboard, click **New**.
2. Choose **PostgreSQL**.
3. Create the database.
4. Copy the database connection string.
5. Open the backend Web Service settings.
6. Add the connection string as the environment variable `DATABASE_URL`.

The backend also needs environment variables for authentication and classroom seeding. Use strong private values instead of the examples below.

Table 10: Suggested Render Environment Variables

Variable	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection string
<code>NODE_ENV</code>	Set to production
<code>JWT_SECRET</code>	Secret used to sign login tokens
<code>SEED_SECRET</code>	Secret used to protect the reseed endpoint, if enabled

After saving the settings, deploy the service. When deployment finishes, Render will provide a backend URL similar to:

`https://acc3202-ticketing-bot.onrender.com`

A.6 Reseeding the Database

Before class, reseed the database so that all tickets, events, accounts, and wallet balances are reset. The classroom command is provided on the teaching page. In PowerShell, the command has the following form:

```
curl.exe -X POST "https://acc3202-ticketing-bot.onrender.com/api/admin/seed-database" `
-H "x-seed-secret: YOUR_PRIVATE_SEED_SECRET"
```

Replace YOUR_PRIVATE_SEED_SECRET with the seed secret used for the class deployment.

A.7 Logging In as Admin

The instructor can log in as the admin user through PowerShell:

```
$response = Invoke-RestMethod `
-Uri "https://acc3202-ticketing-bot.onrender.com/api/login" `
-Method POST `
-ContentType "application/json" `
-Body '{"email":"admin@hunter-ticket.com","password":"adminpass"}'
```

```
$token = $response.token
```

The variable \$token stores the admin login token. The instructor needs this token to release tickets and check holdings.

A.8 Releasing Tickets

To release additional tickets for a specific ticket type, use:

```
Invoke-RestMethod `
-Uri "https://acc3202-ticketing-bot.onrender.com/api/admin/release-more" `
-Method POST `
-ContentType "application/json" `
-Headers @{ Authorization = "Bearer $token" } `
-Body '{"ticketTypeId":5,"additionalQuantity":10}'
```

Change ticketTypeId and additionalQuantity as needed. For the classroom exercise, the instructor should release approximately $N - 20\%$ tickets for each event, where N is the number of students.

For example, if there are 70 students:

$$70 - (70 \times 0.20) = 56$$

Thus, the instructor would release approximately 56 tickets for the event.

A.9 Checking Student Holdings

To check which students purchased tickets, run:

```
Invoke-RestMethod `
  -Uri "https://acc3202-ticketing-bot.onrender.com/api/admin/holdings" `
  -Method GET `
  -Headers @{ Authorization = "Bearer $token" } |
Format-Table email, wallet_balance, event_title, ticket_type, quantity_owned, amount_spent -
AutoSize
```

This command lets the instructor see each student's wallet balance, event title, ticket type, quantity owned, and amount spent.

A.10 Checking Public Ticket Availability

To check whether tickets are released or sold out for an event, run:

```
Invoke-RestMethod `
  -Uri "https://acc3202-ticketing-bot.onrender.com/api/events/2/tickets" `
  -Method GET |
Select-Object -ExpandProperty tickets |
Format-Table id, name, price, releasedQuantity, soldQuantity, availableQuantity, isReleased, soldOut -
AutoSize
```

Change the event number in `/api/events/2/tickets` to inspect a different event.

A.11 Student Bot Program

The following PowerShell script is used only for the sandboxed classroom website. It logs in as a student user, repeatedly checks whether a ticket is available for one event, and purchases one ticket when it becomes available.

```
# Login as a student user
$response = Invoke-RestMethod `
  -Uri "https://acc3202-ticketing-bot.onrender.com/api/login" `
  -Method POST `
  -ContentType "application/json" `
  -Body '{"email":"test@test.com","password":"test"}'

$token = $response.token

# Choose event
```

```

$eventId = 5

# Bot loop: keep checking until ticket is available
while ($true) {
    $ticketInfo = Invoke-RestMethod `
        -Uri "https://acc3202-ticketing-bot.onrender.com/api/events/$eventId/tickets" `
        -Method GET

    $availableTicket = $ticketInfo.tickets |
        Where-Object { $_.isReleased -eq $true -and $_.availableQuantity -gt 0 } |
        Select-Object -First 1

    if ($availableTicket) {
        Write-Host "Ticket found!"
        Write-Host "Ticket Type ID:" $availableTicket.id
        Write-Host "Price:" $availableTicket.price

        $body = @{
            eventId = $eventId
            items = @(
                @{
                    ticketTypeId = $availableTicket.id
                    quantity = 1
                }
            )
        } | ConvertTo-Json -Depth 5

        $purchase = Invoke-RestMethod `
            -Uri "https://acc3202-ticketing-bot.onrender.com/api/purchase" `
            -Method POST `
            -ContentType "application/json" `
            -Headers @{ Authorization = "Bearer $token" } `
            -Body $body

        Write-Host "Purchase complete!"
        $purchase
        break
    }
}

```

```

Write-Host "No ticket yet. Checking again..."
Start-Sleep -Milliseconds 500
}

```

Students should be reminded that this script is authorized only for the sandboxed classroom site. It must not be used on real ticketing websites, commercial platforms, or systems for which the student lacks explicit permission.

A.12 Keeping the Render Site Active During Class

Because the Render backend is on a free server, it may become inactive after a period without traffic. To reduce delays during class, the instructor can create a GitHub Actions workflow that visits the Render backend every five minutes.

Create a file in the repository at:

```
.github/workflows/keep-render-awake.yml
```

Add the following:

```

name: Keep Render Backend Awake

on:
  schedule:
    - cron: "* /5 * * * *"
  workflow_dispatch:

jobs:
  ping:
    runs-on: ubuntu-latest
    steps:
      - name: Ping Render backend
        run: |
          curl -I https://acc3202-ticketing-bot.onrender.com || true

```

This workflow sends a small request to the backend every five minutes. The `workflow_dispatch` line also lets the instructor manually run the workflow from the GitHub Actions tab.

A.13 Alternative Keep-Alive Option

Instead of GitHub Actions, the instructor can use an external cron service to visit the backend URL every five minutes during the class session:

```
https://acc3202-ticketing-bot.onrender.com
```

The instructor should disable the keep-alive job when the exercise is finished if the site does not need to remain active.

A.14 Before Class Checklist

Before the exercise begins, the instructor should complete the following steps:

1. Update the student list in `server/seed.js`.
2. Deploy the backend on Render.
3. Create or connect the PostgreSQL database.
4. Set the required Render environment variables.
5. Reseed the database.
6. Confirm that Event 1 has \$0 tickets.
7. Confirm that the remaining four events have \$1 tickets.
8. Confirm that each student can log in.
9. Confirm that the admin login works.
10. Confirm that ticket release commands work.
11. Confirm that the holdings command displays student purchases.
12. Confirm that the keep-alive cron job is active during class.
13. Open the instructional video and classroom command page for reference.

B Instructor Notes

The case should be positioned carefully. The botting activity is not the educational endpoint. It is the controlled event that reveals why internal controls matter.

The central teaching point is that automated behavior can expose whether a digital transaction system truly enforces its stated rules. IT auditors must understand both the technical mechanics of the system and the internal-control consequences of those mechanics.

The exercise proceeds in three stages. First, students participate in a limited ticket release using only ordinary manual interaction. This creates a baseline experience of scarcity and transaction speed. Second, students inspect the sandboxed website using browser developer tools and identify the DOM elements that govern the search, event-selection, and purchase process. Third, the instructor demonstrates how automated interaction can exploit weak front-end controls. Students then evaluate the same activity from the perspective of an IT auditor by testing purchase-limit controls, rate-limiting controls, logging, inventory integrity, and exception reporting.

The purpose of the botting exercise is not to train students to scalp tickets. Rather, the purpose is to show how automated behavior can expose weaknesses in digital internal controls. By requiring students to inspect the DOM, observe automated interaction, test purchase-limit controls, analyze exception logs, and prepare audit documentation, the case connects technical fluency with traditional audit judgment.

Observed Weakness	Possible Evidence from Exercise or Backend Log	Recommended Control	Control Type
Tickets disappeared too quickly for ordinary users	Most tickets were purchased within a short time window	Queue system, randomized waiting room, or rate-limited checkout	Preventive
Script could target predictable page elements	Purchase buttons or event selectors used stable IDs, classes, or selectors	Server-generated transaction tokens or dynamic checkout validation	Preventive
Purchase process relied heavily on front-end behavior	Browser-side actions triggered transaction attempts directly	Server-side validation of every purchase request	Preventive
System did not flag rapid purchases	Repeated attempts appeared as ordinary transactions	Bot exception report based on speed, repetition, and failed attempts	Detective
Audit trail was incomplete	Backend log lacked user ID, session ID, timestamp, event ID, ticket tier, or request source	Complete transaction logging with an immutable audit trail	Detective
No exception review process	Suspicious transactions were logged but not summarized for management review	Post-drop exception report reviewed and signed off by management	Detective